

UNIVERSITÉ DU QUÉBEC À MONTRÉAL

TRAVAIL PRÉSENTÉ À

Flavio castells

DANS LE CADRE DU COURS

TESTS LOGICIELS (MGL7230)

PAR

MOHAMED DJAWAD ABI AYAD

ABIM14288800

1. Rapport de Refactorisation et Améliorations du Code

a. Refactorisation de la classe Application

Dans la première version du code, la classe Application regroupait plusieurs responsabilités, notamment l'initialisation des services, la gestion de la logique métier et le traitement des erreurs. Cette centralisation rendait le code moins lisible et plus complexe à maintenir.

Dans la seconde version, une refonte a été effectuée pour améliorer la séparation des préoccupations, aboutissant à une structure plus modulaire avec trois classes statiques distinctes :

- **Initializer** : Responsable de l'instanciation des services et de la récupération des informations initiales.
- **Runner** : Gère la logique métier, notamment l'ajout des passagers et la réservation des vols.
- **Application** : Point d'entrée du programme, qui prend en charge la gestion des exceptions lors de l'ouverture du fichier.

Cette restructuration apporte plusieurs bénéfices : elle améliore la clarté et la maintenabilité du code en répartissant les responsabilités de manière cohérente, tout en rendant l'ensemble plus robuste face aux erreurs.

Une modification importante concerne la gestion des exceptions. Désormais, la gestion des erreurs liées à l'ouverture du fichier a été déplacée dans la méthode main. Cette approche permet de laisser le fichier ouvert pour l'écriture pendant toute l'exécution du programme, garantissant une meilleure gestion des ressources. De plus, les ressources ne sont fermées qu'à la fin du programme, évitant ainsi des interruptions prématurées. Cette amélioration a été suggérée par SonarQube, qui a identifié un problème potentiel dans la première version du code.

b. Refactorisation de la classe Runner

La refactorisation de la classe Runner a permis d'améliorer la gestion des passagers, des sièges et des ressources. Un contrôle des doublons a été ajouté, empêchant l'enregistrement du même passager sur le même vol en vérifiant leur numéro de passeport.

Le statut du vol est désormais mis à jour à FULL dès que les sièges sont épuisés, ce qui bloque l'ajout de passagers une fois le vol complet. De plus, l'instanciation des services a été déplacée en dehors de la boucle, optimisant ainsi les performances en évitant les appels redondants. Enfin, la logique d'arrêt du processus a été améliorée, permettant l'arrêt automatique de l'ajout de passagers lorsque le vol est plein, tout en laissant l'utilisateur continuer si nécessaire.

c. Refactorisation de BookingService

Les améliorations apportées rendent le processus de gestion des passagers plus lisible et plus maintenable. La nouvelle version sépare clairement le traitement des passagers en fonction de leur classe (**First, Business, Economy**) grâce à des méthodes dédiées.

Cela rend le code plus compréhensible et plus facile à étendre. De plus, la gestion des erreurs et des transitions entre les classes de sièges est désormais plus cohérente. L'utilisation de messages de journalisation permet également de suivre plus facilement le déroulement du processus, et l'élimination de la redondance améliore la maintenabilité du code.

d. Refactorisation de PassengerService

Les améliorations optimisent le processus de création du passager en validant immédiatement la classe du passager. En cas de classe invalide, le code retourne immédiatement null, éliminant ainsi une boucle inutile.

L'utilisation d'un switch pour la gestion des différentes classes de passagers améliore la lisibilité et facilite l'ajout de nouvelles classes à l'avenir. Cette réorganisation du code améliore sa maintenabilité, sa lisibilité, et réduit le risque d'erreurs.

e. Ajout de l'état à la classe FlightInformation

Les modifications incluent l'initialisation automatique de l'attribut `flightStatus` à OPEN lors de l'instanciation d'un vol, garantissant ainsi un état initial cohérent. De plus, les méthodes de gestion de l'état du vol, telles que `setFlightStatus`, permettent de gérer dynamiquement les changements d'état, notamment le passage à FULL.

f. Redéfinition de la méthode equals() pour la classe Passenger

L'ajout de la méthode `equals()` permet de comparer les passagers sur la base de leur numéro de passeport, servant d'identifiant unique. Cette méthode renforce la cohérence des structures de données, évitant ainsi les doublons.

g. Autres améliorations

- **Remplacement de `System.out.println` par un logger** : Améliore la gestion des journaux en permettant un meilleur suivi des événements, une distinction des niveaux de log et une redirection plus efficace des messages.
- **Remplacement des valeurs en dur par des constantes** : Améliore la lisibilité, la maintenabilité et réduit le risque d'erreur.

Ces modifications renforcent la qualité du code en le rendant plus modulaire, évolutif et conforme aux bonnes pratiques recommandées par SonarQube.

1. Rapport sur les tests unitaires et l'utilisation du mock

Afin de tester une application qui n'a pas encore été validée, des tests unitaires ont été réalisés en utilisant la technique du mock des dépendances. Cette approche permet de localiser avec précision toute erreur logique lors des modifications.

En cas d'échec des tests unitaires, seul le test concerné échoue, sans affecter les autres tests dépendant de différentes classes.

Les tests unitaires ont été structurés en plusieurs étapes :

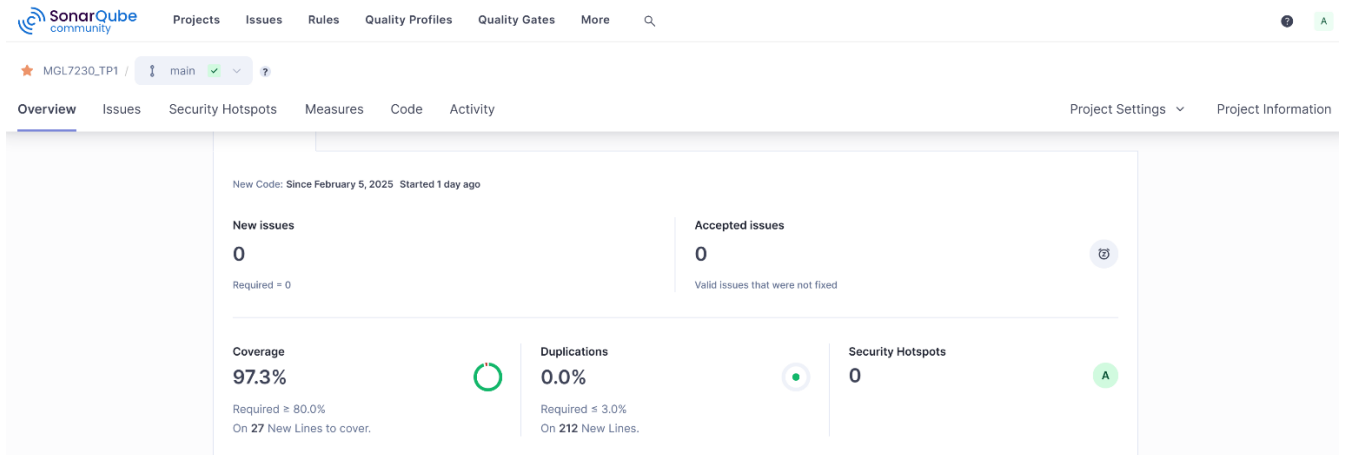
1. **Tests sur les classes de service** : Simulation des opérations de lecture et d'écriture sur la couche de persistance.
2. **Tests des services métier** : Vérification des fonctionnalités critiques.
3. **Tests de la classe principale** : Validation de l'initialisation des instances et de la gestion de la logique métier.

Dans le cadre de l'optimisation des tests sur la classe principale, une refactorisation a été réalisée en la scindant en trois classes statiques. Cette modification a permis l'utilisation de **Mockito Inline**, facilitant ainsi la vérification des appels aux méthodes statiques.

Mockito Inline a également été utilisé pour intercepter les objets instanciés dynamiquement et non injectés via un constructeur ou en paramètre. Grâce à cette approche, ces objets ont pu être mockés avec des valeurs de retour spécifiques, garantissant ainsi des tests unitaires précis et reproductibles.

Cette méthodologie assure une validation rigoureuse de l'application en isolant les différents composants et en garantissant que chaque module fonctionne correctement avant leur intégration finale.

2. Rapport de l'analyse statique de sonarQube avec pourcentage de couverture des test



MGL7230_TP1 > src > main/java/ca/uqam/mgl7230/tp1								
	Lines of Code	Security	Reliability	Maintainability	Security Hotspots	Coverage	Duplications	
main/java/ca/uqam/mgl7230/tp1	651	0	0	0	0	96.3%	0.0%	
adapter	85	0	0	0	0	100%	0.0%	
config	16	0	0	0	0	100%	0.0%	
exception	6	0	0	0	0	100%	0.0%	
model	167	0	0	0	0	96.2%	0.0%	
service	248	0	0	0	0	95.5%	0.0%	
utils	13	0	0	0	0	100%	0.0%	
Application.java	29	0	0	0	0	93.3%	0.0%	
Initializer.java	26	0	0	0	0	90.9%	0.0%	
Runner.java	61	0	0	0	0	97.6%	0.0%	

9 of 9 shown